The University of Texas at Austin
Chandra Department of Electrical and Computer Engineering
Cockrell School of Engineering

APRIL 2026

HaaS Backend Testing

DAVID CHO, EDUARDO ROSALES, and ANITA WOOFORD
ECE 382V, The University of Texas at Austin

Motivation and Research Question (RQ)

- Backend systems are tested under time pressure; developers write scenario-based tests by intuition, but intuition alone does not systematically cover the input space or logic branches
- Formal testing criteria (ISP, prime path coverage, CACC, mutation analysis) provide repeatable, criterion-driven processes for deriving tests, but their benefit over a developer-style baseline is not always obvious
- **RQ**: To what extent do formal structural and logic-based coverage criteria (prime path, CACC, input space partitioning, and mutation analysis) improve defect detection effectiveness and coverage metrics compared to a constrained baseline test suite?

.

Hardware as a Service (HaaS) Background

- Testing suites have been based on a HaaS backend from Native Cloud Development class; backend is a cloud-native REST API built with Flask and backed by MongoDB
- Provides 3 core capabilities: user auth, hardware inventory management, and project management
- Testing focused on 3 targets across 2 route files:
  1. _encrypt(): cyclic Caesar cipher used to hash passwords before storage
  2. login() / register(): credential validation and user creation
  3. checkout() / checkin(): hardware inventory allocation and return endpoints

.

Shared Infrastructure and Baseline Test Suite Rules

- Baseline design rules:
  1. Baseline design limited to a fixed effort budget (4 tests per function/endpoint)
  2. Common scenario template: 1 happy-path case, one common invalid-input case, one common missing-field or unauthorized case when applicable, and 1 boundary, guard, or behavioral case
  3. Tests are not derived from CFGs, path enumeration, or logic-coverage tables
  4. Baseline and criteria-based suites must use the infrastructure/mocking approach
  5. Baseline tests are required to include meaningful assertions over status codes, responses, or persisted state
- Infrastructure:
  - All test suites share a common in-memory test double replacing MongoDB:
    - FakeCollection: implements find_one, insert_one, update_one...etc with MongoDB operator support ($set, $inc, $pull, etc.)
    - FakeDB: routes collection lookups by name (users, hardware, projects)

.

_encrypt() Target Overview

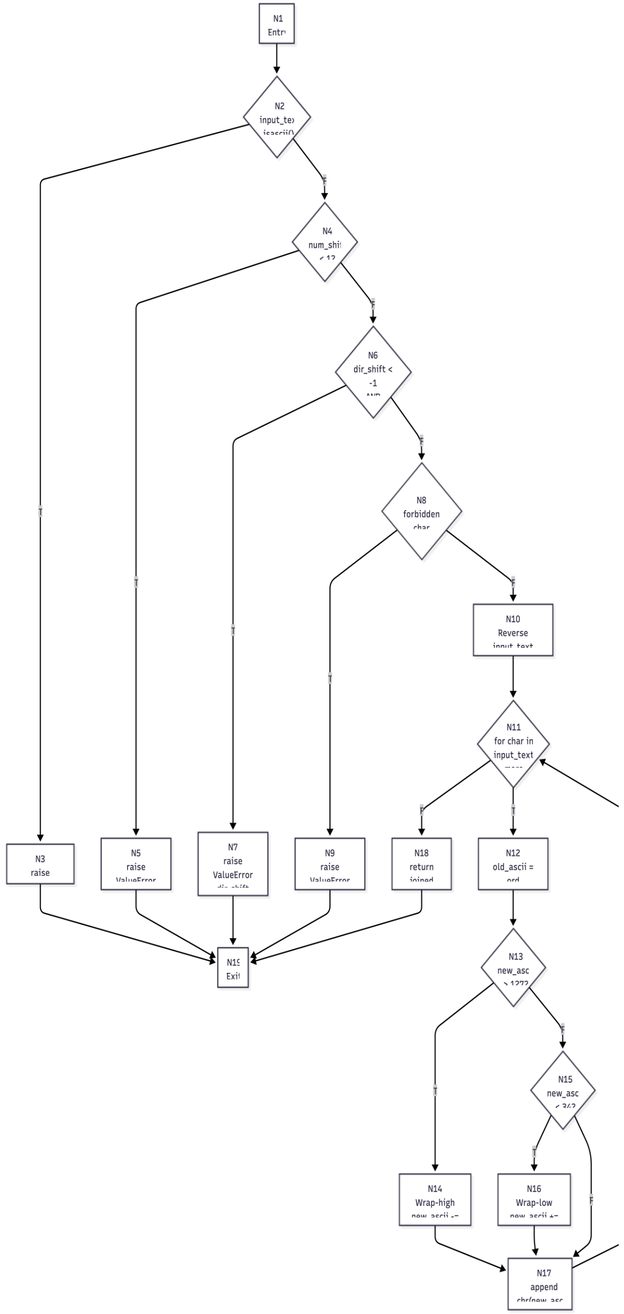
- Function _encrypt() (auth.py)
- Purpose: Cyclic ASCII shift cipher
- Inputs:
  - Input_text(string)
  - Num_shift(int)
  - Dir_shift(±1 expected)
- Behavior:
  - Input validation
  - ASCIIU transformation
  - Wrap around logic
- Techniques:
  - CFG(Graph Coverage)
  - CACC(Logic Coverage)
  - Input Domain
  - Mutation Testing(RIP)

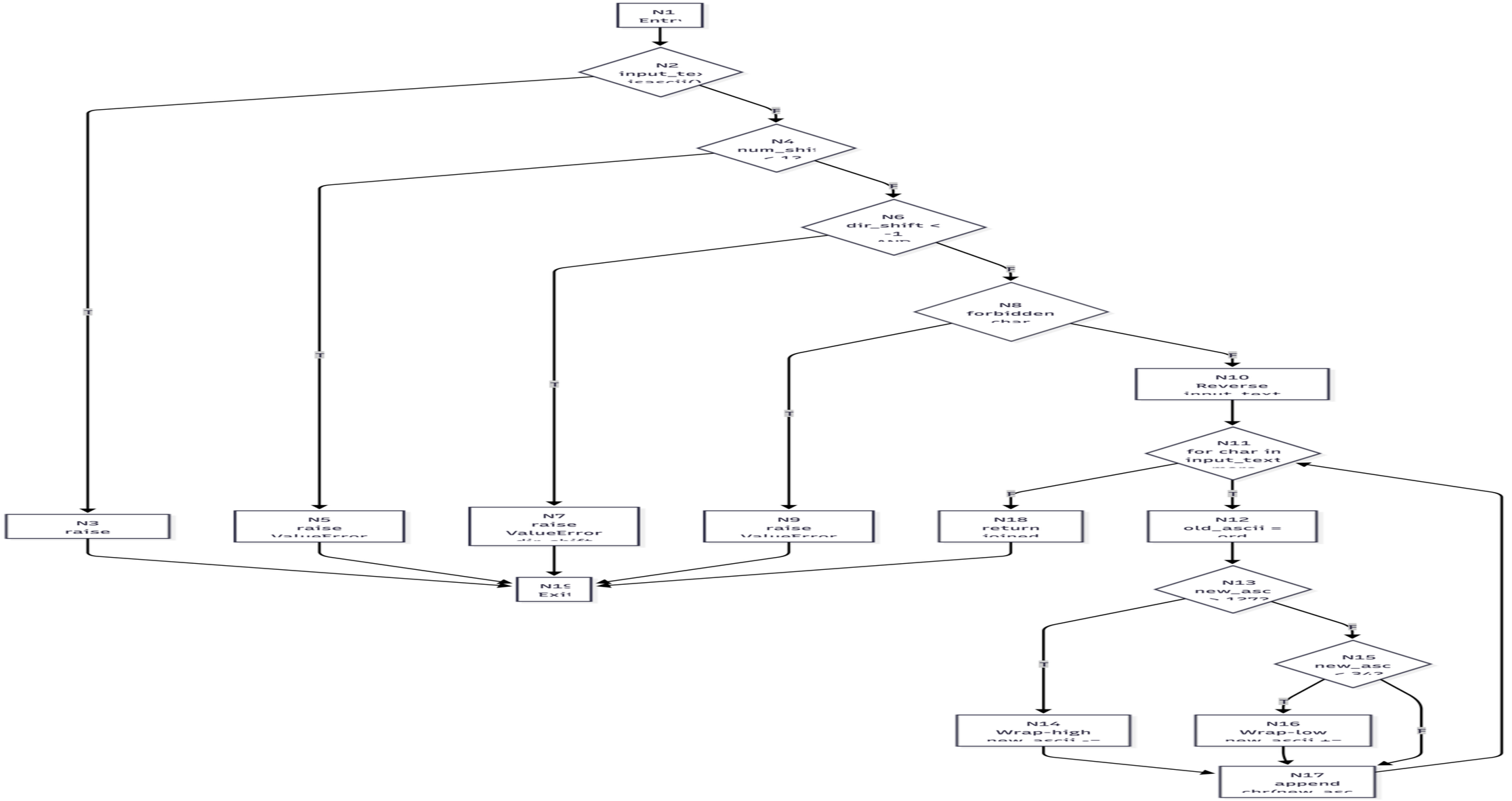
Goal: evaluate test effectiveness using formal testing vs baseline.

.

CFG and Test Requirements

- 19 feasible nodes(1 infeasible)
- 21 Edges (20 feasible)
- 6 Decision Branches
  - B1: ASCII Check
  - B2: num_shift < 1
  - B3: dir_shift predicate
  - B4: forbidden chars
  - B5: wrap high
  - B6: wrap low
- Test Requirements
  - Node Coverage
  - Edge Coverage
  - Prime Path Coverage




N1
N2
N4
N6
N8
N10
N11
N3
raise
N5
raise
N7
raise
ValueError
N9
raise
N18
return
N12
N13
N15
N14
Wrap-high
N16
Wrap-low
N17
append
T
F

CFG Nodes

Node	Branch	Description
N1	–	start
N2	B1	input_text.isascii()?
N3	–	Raise TypeError
N4	B2	num_shift <1?
N5	–	Raise ValueError (num_shift)
N6	B3	dir_shift < -1 AND dir_shift > 1? (unsatisfiable predicate BUG)
N7	–	Raise ValueError(dir_shift)(infeasible: dead code due to unsatisfiable predicate)
N8	B4	forbidden char present?
N9	–	Raise ValueError(forbidden char)
N10	–	Reverse string and init loop
N11	–	Loop condition(for char in input_text)
N12	–	Compute new_ascii
N13	B5	new_ascii > 127?
N14	–	Wrap-around-high(new_ascii -=128)
N15	B6	new_ascii < 34
N16	–	Wrap-around-low(new_ascii += 128)
N17	–	Append shifted char
N18	–	Return encrypted string
N19	–	End

Edges

Edge	Definition
E1	(N1 → N2)
E2	(N2 True → N3)
E3	(N2 False → N4)
E4	(N4 True → N5)
E5	(N4 False → N6)
E6	(N6 True → N7) infeasible
E7	(N6 False → N8)
E8	(N8 True → N9)
E9	(N8 False → N10)
E10	(N10 → N11)
E11	(N11 True → N12)
E12	(N11 False → N18)
E13	(N12 → N13)
E14	(N13 True → N14)
E15	(N13 False → N15)
E16	(N14 → N17)
E17	(N15 True → N16)
E18	(N15 False → N17)
E19	(N16 → N17)
E20	(N17 → N11)
E21	(N18 → N19)

Branch Definitions

Branch	Definition
B1	input_text.isascii()
B2	num_shift < 1
B3	dir_shift < -1 AND dir_shift > 1 (infeasible)
B4	forbidden character present
B5	new_ascii > 127
B6	new_ascii < 34

Path	Definition
P1	N1 -> N2 -> N3 (non-ASCII Error path)
P2	N1 -> N2 -> N4 -> N5 (Error path num_shift)
P3	N1 -> N2 -> N4 -> N6 -> N8 -> N9 (forbidden char Error path)
P4	N10 -> N11 -> N12 -> N13 -> N15 -> N17 Valid path no-wrap
P5	N10 -> N11 -> N12 -> N13 -> N14 -> N17 Valid path wrap-high
P6	N10 -> N11 -> N12 -> N13 -> N15 -> N16 -> N17 Valid path wrap-low
P7	N10 -> N11 -> N18 -> N19 (Loop exit path when false)
P8	N6 -> N7 (Infeasible path through N6 True (excluded)) dir_shift < -1 AND dir_shift > 1 is unsatisfiable

Infeasible Branch- B3

- Predicate :
  - `dir_shift < -1 AND dir_shift > 1`
- Define Clauses:
  - A = (dir_shift < -1)
  - B = (dir_shift > 1)
- Result:
  - Always FALSE
  - Node N7 unreachable
  - Edge E6 excluded
- CACC Result
  - No Clause determines predicate
  - Predicate is unsatisfiable

dir_shift	A (< -1)	B (> 1)	P = A AND B
-2	TRUE	FALSE	FALSE
0	FALSE	FALSE	FALSE
2	FALSE	TRUE	FALSE

Structural Test Coverage

- Structural suite: 9/9 passed
- Feasible coverage
  - Node Coverage: 100% feasible
  - Edge Coverage: 100 % feasible
  - Prime Path Coverage: 100% feasible

CACC result:
Analyzed predicate in dir_shift validation
true branch shown infeasible by construction

Infeasible branch was excluded.

Test Mapping Evidence

Test Case	Description	CFG Edges Covered	Prime Paths Covered
T1	Non-ASCII input	E1, E2	P1
T2	num_shift < 1	E1, E3, E4	P2
T3	Forbidden character	E1, E3, E5, E7, E8	P3
T4	Valid input (no wrap)	E10–E21 (loop execution edges)	P4
T5	Wrap-around-high condition	—	P5
T6	Wrap-around-low condition	—	P6
T7	Empty string (loop exit)	—	P7
T8	Loop cycle (single iteration)	E20 (loop back edge)	—
T9	Multiple loop iterations	E20 (loop back edge, repeated execution)	—

Input Domain Model

- Characteristics
  - Input_text
  - Num_shift
  - dir_shift
- Partitions
  - ASCII / non-ASCII/ forbidden/wrap/empty
  - Num_shift(<1,=1, >1)
  - Dir_shift (=1,-1, invalid)
- Base choice coverage
- 8 test cases / 8 passed
- Results:
  - Invalid dir_shift not rejected ->defect
- Conclusio: partitions systematically exercised.

Base Choice Test Mapping

Test	C1	C2	C3	Input Values	Expected Result
BT (Base)	b1	b2	b1	("mypassword", 1, 1)	Valid encrypted string
T1	b2	b2	b1	("mypasswordé", 1, 1)	TypeError
T2	b3	b2	b1	("pass word!#", 1, 1)	ValueError
T3	b4	b2	b1	("~", 1, 1)	Valid (wrap-high)
T4	b5	b2	b1	("#", 1, 1)	Valid (wrap-low)
T5	b1	b1	b1	("mypassword", 0, 1)	ValueError
T6	b1	b2	b3	("mypassword", 1, 2)	BUG (should ValueError)
T7	b6	b2	b1	("", 1, 1)	Valid (empty string)

Mutation Testing

- Mutation testing measured fault sensitivity for the function
- Manual mutants targed
  - Boundary conditions
  - Logic operator behavior
  - 4 mutants created/ 4 killed
  - Results: 100% mutation score

- RIP:
  - Reachable -> test executes condition
  - Infection -> incorrect state created
  - Propagation -> output differs -> failure observerd

Mutants Generated

Four manual mutants were created targeting different code regions:

Mutant ID	Mutation Type	Location	Change	Rationale
M1	Boundary condition	Line 27 (num_shift validation)	num_shift < 1 → num_shift < 0	Tests boundary off-by-one detection
M2	Logical operator	Line 26 (dir_shift validation)	AND → OR in predicate	Tests logical operator sensitivity
M3	Relational operator (wrap-high)	Line 45 (wrap-high threshold)	new_ascii > 127 → new_ascii >= 127	Tests wrap-around boundary detection
M4	Relational operator (wrap-low)	Line 51 (wrap-low threshold)	new_ascii < 34 → new_ascii <= 34	Tests wrap-around boundary detection

RIP Worksheet Summary

All four mutants were successfully killed by the test suite:

Mutant	Kill Method	Test Case	Evidence	RIP Status
M1	num_shift boundary	T2, T4	ValueError raised for num_shift=0 (mutation expects pass)	Killed
M2	dir_shift logic	T3, T4	Predicate now satisfiable; test catches invalid dir_shift values	Killed
M3	wrap-high boundary	T5	Incorrect wrap-around ASCII value (off-by-one); test detects mismatch	Killed
M4	wrap-low boundary	T6	Incorrect wrap-around ASCII value (off-by-one); test detects mismatch	Killed

Defect Found

- Documented Defect:
  - Unsatisfiable predicate in dir_shift validation
  - Uses and between mutually exclusive bounds
- Effect
  - Invalid dir_shift values can bypass intended rejection path
- Recommend Patch
  - Change dir_shift <-1 and dir_shift > 1
  - To dir_shift <-1 or dir_shift > 1

Results

- Final Result
  - 1 defect identified(D-7)
  - Infeasilbe branch proven
  - 100% structural coverage
- Formal methods exposed:
  - Unreachable logic
  - Correctness gaps beyond baseline
- _encrypt campaign progression
  - Baseline around 42%
  - Structural/CACC/partition around 40%
  - Combined campaign up to 88%

- Combined _encrypt campaign: 61 test passed

Tests distinguish the program from faulty variants.

Mutants Generated

Four manual mutants were created targeting different code regions:

Mutant ID	Mutation Type	Location	Change	Rationale
M1	Boundary condition	Line 27 (num_shift validation)	num_shift < 1 → num_shift < 0	Tests boundary off-by-one detection
M2	Logical operator	Line 26 (dir_shift validation)	AND → OR in predicate	Tests logical operator sensitivity
M3	Relational operator (wrap-high)	Line 45 (wrap-high threshold)	new_ascii > 127 → new_ascii >= 127	Tests wrap-around boundary detection
M4	Relational operator (wrap-low)	Line 51 (wrap-low threshold)	new_ascii < 34 → new_ascii <= 34	Tests wrap-around boundary detection

RIP Worksheet Summary

All four mutants were successfully killed by the test suite:

Mutant	Kill Method	Test Case	Evidence	RIP Status
M1	num_shift boundary	T2, T4	ValueError raised for num_shift=0 (mutation expects pass)	Killed
M2	dir_shift logic	T3, T4	Predicate now satisfiable; test catches invalid dir_shift values	Killed
M3	wrap-high boundary	T5	Incorrect wrap-around ASCII value (off-by-one); test detects mismatch	Killed
M4	wrap-low boundary	T6	Incorrect wrap-around ASCII value (off-by-one); test detects mismatch	Killed

- Formal methods exposed:
  - Unreachable logic
  - Correctness gaps beyond baseline

register() + login() Target Overview

- Responsible for credential storage and validation
- API endpoints:
  1. POST /api/auth/register
  2. POST /api/auth/login
- Has known defect
  - Checks whether userId exists in the dictionary but not the userId itself

ISP Characteristics and Blocks

- Built input domain models (IDMs) for register() and login() separately
- Base choice: <b1, b1, b1> is a standard happy path
- Non-base frames target one block deviation at a time (Base choice coverage)

Characteristic	Block	Input	Expected	Observed	Diverges
C1 - userId field value	b1 - Non-empty string	Valid string (e.g. "alice")	201	201	No
C1 - userId field value	b2 - Key omitted	No userId key in body	400	400	No
C1 - userId field value	b3 - Empty string	"userId": ""	400	201	Yes
C1 - userId field value	b4 - Null	"userId": null	400	201	Yes
C2 - password field value	b1 - Non-empty string	Valid string (e.g. "secret")	201	201	No
C2 - password field value	b2 - Key omitted	No password key in body	400	400	No
C2 - password field value	b3 - Empty string	"password": ""	400	201	Yes
C2 - password field value	b4 - Null	"password": null	400	500	Yes
C3 - userId uniqueness	b1 - New user	userId not in database	201	201	No
C3 - userId uniqueness	b2 - Duplicate	userId already registered	409	409	No

ISP Defects Found

- 4 defects in register(), 2 in login()
- e.g., D-1: userId in data passes for {“userId”: “”} because the key exists

ID	Target	Input / Condition	Intended	Observed	Suite
D-1	register()	userId: ""	400	201 — empty string stored as userId	ISP (C1:b3)
D-2	register()	userId: null	400	201 — null stored as userId	ISP (C1:b4)
D-3	register()	password: ""	400	201 — empty password stored	ISP (C2:b3)
D-4	register()	password: null	400	500 — unhandled crash (AttributeError)	ISP (C2:b4)
D-5	login()	userId: ""	400	401 — misclassified as invalid credentials	ISP (C1:b3)
D-6	login()	password: ""	400	401 — misclassified as invalid credentials	ISP (C2:b3)
D-7	_encrypt()	dir_shift: 2	ValueError	No error raised	ISP (C3:b3); Structural (infeasible branch); CACC (unsatisfiable predicate)

register() + login() Mutation Scores

- Mutation scores: Baseline 63.6%, Partition 62.0%
- Mutant 56:
  - Survived both suites – MagicMock.__getitem__ returned the same collection for any key, so the wrong-collection mutation was never detected
- Partition suite killed mutants tied to the empty-string path; baseline does not reach that code
- Scores are close numerically but qualitatively different in which mutants survive

register() + login() Results Summary

- Baseline: 8 tests, 0 defects caught
- ISP: 6 defects exposed by forcing empty-string blocks
- Root cause is line 101’s validation logic
- Formal criteria made the gap systematic and reproducible

checkout_hardware() & checkin_hardware()

- Testing the checkout/in functions.
- Techniques:
  - Graph Coverage
  - Logic Coverage
  - Input Partitioning
- Results- based from using coverage analysis

.

Checkout/checkin Overview

Inputs:

- hardware_id

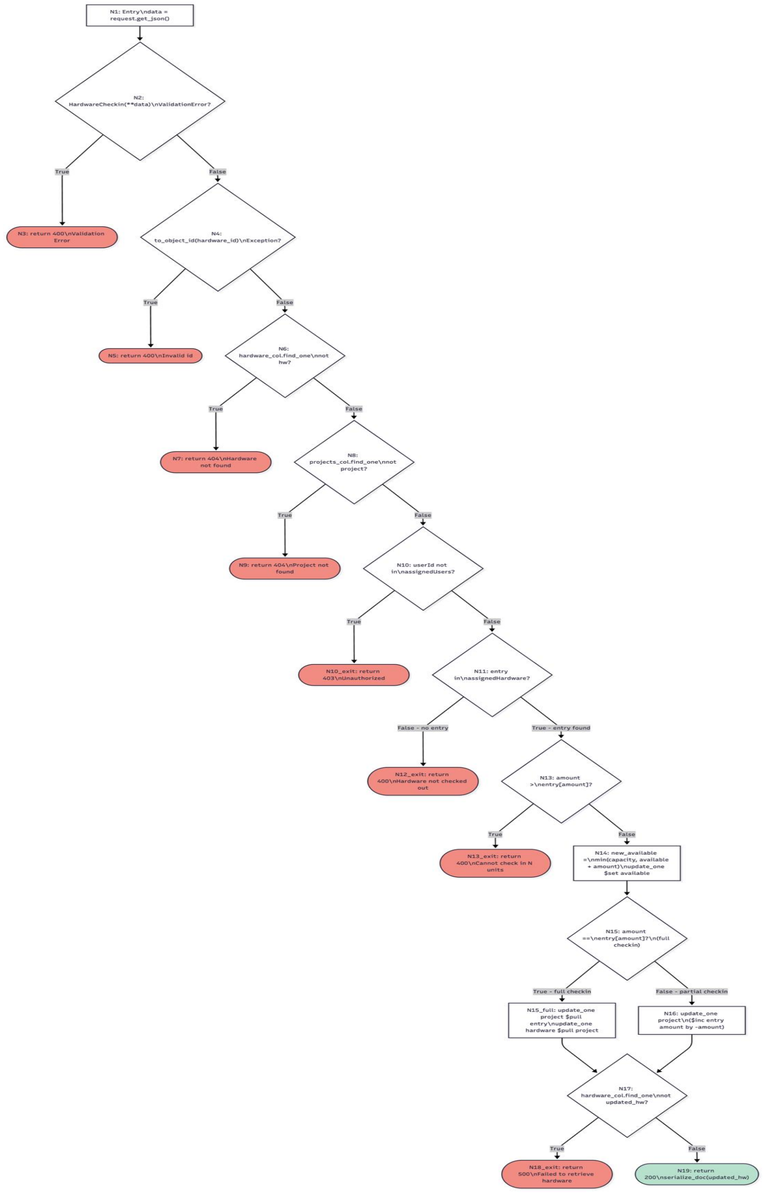
Performs

- validation
- predicate coverage

9 Decision Branches

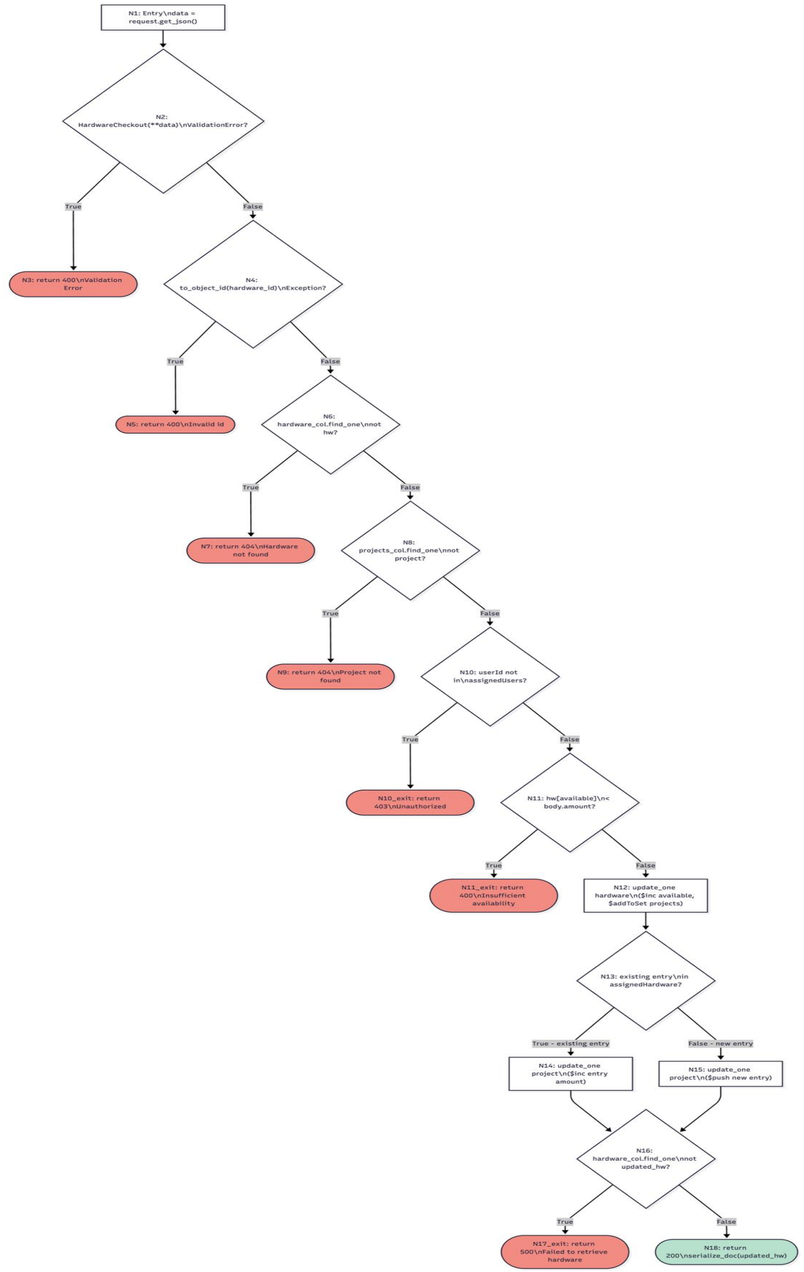
Control Flow Graph for checkin

- 19 Nodes
- 21 Edges for control flow
- 9 Branches



Control Flow Graph for checkout

- 18 Nodes
- 19 Edges for control flow
- 9 Branches



Checkout/in Infeasible Branch

- Predicate
  - p = (amount > 0) AND (amount <= bound)
- Clauses:
  - C1 = (amount > 0)
  - C2 = (amount <= bound)
  - C1 as major clause is behind the infeasible branch

Checkin Graph Coverage

Node	Branch	Description
N1	–	Entry — `data = request.get_json() or {}`
N2	B1	`HardwareCheckin(**data)` raises ValidationError?
N3	–	**[FINAL]** return 400 Validation Error
N4	B2	`to_object_id(hardware_id)` raises Exception?
N5	–	**[FINAL]** return 400 `{"error": "Invalid id"}`
N6	B3	`hardware_col().find_one({"_id": _id})` -> not hw?
N7	–	**[FINAL]** return 404 `{"error": "Hardware not found"}`
N8	B4	`projects_col().find_one({"projectId": …})` -> not project?
N9	–	**[FINAL]** return 404 `{"error": "Project not found"}`
N10	B5	`userId not in project["assignedUsers"]` ?
N10_exit	–	**[FINAL]** return 403 Unauthorized
N11	B6	`entry = next(assignedHardware where hardwareId==hw_id, None)` -> not entry?
N12_exit	–	**[FINAL]** return 400 Hardware not checked out
N13	B7	`amount > entry["amount"]` ? (over-checkin guard)
N13_exit	–	**[FINAL]** return 400 Cannot check in N units
N14	–	`new_available = min(capacity, available + amount)` ; `update_one $set available`
N15	B8	`amount == entry["amount"]` ? (full checkin)
N15_full	–	`update_one project $pull entry` ; `update_one hardware $pull project`
N16	–	`update_one project: $inc assignedHardware.$.amount` by `-amount`
N17	B9	`hardware_col().find_one({"_id": _id})` -> not updated_hw?
N18_exit	–	**[FINAL]** return 500 Failed to retrieve hardware
N19	–	**[FINAL]** return 200 `serialize_doc(updated_hw)`

Edge	Definition	Notes
E1	N1 -> N2	
E2	N2 True -> N3	ValidationError raised
E3	N2 False -> N4	Validation passes
E4	N4 True -> N5	ObjectId parse failure
E5	N4 False -> N6	Valid ObjectId
E6	N6 True -> N7	Hardware document missing
E7	N6 False -> N8	Hardware found
E8	N8 True -> N9	Project document missing
E9	N8 False -> N10	Project found
E10	N10 True -> N10_exit	User not authorized
E11	N10 False -> N11	User authorized
E12	N11 False -> N12_exit	Hardware not checked out for project
E13	N11 True -> N13	Entry found in assignedHardware
E14	N13 True -> N13_exit	Requested return exceeds checked-out amount
E15	N13 False -> N14	Return amount is valid
E16	N15 True -> N15_full	Full checkin — remove entry and project reference
E17	N15 False -> N16	Partial checkin — decrement entry amount
E18	N15_full -> N17	
E19	N16 -> N17	
E20	N17 True -> N18_exit	Post-update fetch failed (infeasible without mock)
E21	N17 False -> N19	Updated hardware fetched

Path	Node Sequence	Description
PP1	N1->N2(T)->N3	ValidationError at body parse
PP2	N1->N2(F)->N4(T)->N5	Invalid ObjectId
PP3	N1->N2(F)->N4(F)->N6(T)->N7	Hardware not found
PP4	N1->N2(F)->N4(F)->N6(F)->N8(T)->N9	Project not found
PP5	N1->N2(F)->N4(F)->N6(F)->N8(F)->N10(T)->N10_exit	Unauthorized user
PP6	N1->...->N10(F)->N11(F)->N12_exit	Hardware not checked out for project
PP7	N1->...->N10(F)->N11(T)->N13(T)->N13_exit	Over-checkin guard fires
PP8	N1->...->N13(F)->N14->N15(T)->N15_full->N17(T)->N18_exit	Full checkin; post-update fetch fails (mock required)
PP9	N1->...->N13(F)->N14->N15(T)->N15_full->N17(F)->N19	Full checkin succeeds; entries removed
PP10	N1->...->N13(F)->N14->N15(F)->N16->N17(T)->N18_exit	Partial checkin; post-update fetch fails (mock required)
PP11	N1->...->N13(F)->N14->N15(F)->N16->N17(F)->N19	Partial checkin succeeds; entry decremented

Infeasible branches: N17->N18_exit (PP8, PP10) share the same infeasibility as checkout. Both tested via counter-based mock.

Checkout Graph Coverage

Node	Branch	Description
N1	–	Entry – `data = request.get_json() or {}`
N2	B1	`HardwareCheckout(**data)` raises ValidationError?
N3	–	**[FINAL]** return 400 Validation Error
N4	B2	`to_object_id(hardware_id)` raises Exception?
N5	–	**[FINAL]** return 400 `{"error": "Invalid id"}`
N6	B3	`hardware_col().find_one({"_id": _id})` -> not hw?
N7	–	**[FINAL]** return 404 `{"error": "Hardware not found"}`
N8	B4	`projects_col().find_one({"projectId": …})` -> not project?
N9	–	**[FINAL]** return 404 `{"error": "Project not found"}`
N10	B5	`userId not in project["assignedUsers"]` ?
N10_exit	–	**[FINAL]** return 403 Unauthorized
N11	B6	`hw["available"] < body.amount` ?
N11exit	–	**[FINAL]** return 400 Insufficient availability
N12	–	`update_one` hardware: `$inc available` , `$addToSet assignedProjects`
N13	B7	`existing_entry` found in `assignedHardware` ?
N14	–	`update_one` project: `$inc assignedHardware.$.amount`
N15	–	`update_one` project: `$push` new entry
N16	B8	`hardware_col().find_one({"_id": _id})` -> not updated_hw?
N17exit	–	**[FINAL]** return 500 Failed to retrieve hardware
N18	–	**[FINAL]** return 200 `serialize_doc(updated_hw)`

Path	Node Sequence	Description
PP1	N1->N2(T)->N3	ValidationError at body parse
PP2	N1->N2(F)->N4(T)->N5	Invalid ObjectId
PP3	N1->N2(F)->N4(F)->N6(T)->N7	Hardware not found
PP4	N1->N2(F)->N4(F)->N6(F)->N8(T)->N9	Project not found
PP5	N1->N2(F)->N4(F)->N6(F)->N8(F)->N10(T)->N10_exit	Unauthorized user
PP6	N1->N2(F)->N4(F)->N6(F)->N8(F)->N10(F)->N11(T)->N11_exit	Insufficient availability
PP7	N1->...->N11(F)->N12->N13(T)->N14->N16(T)->N17_exit	Existing entry; post-update fetch fails (mock required)
PP8	N1->...->N11(F)->N12->N13(T)->N14->N16(F)->N18	Existing entry; checkout succeeds ($inc path)
PP9	N1->...->N11(F)->N12->N13(F)->N15->N16(T)->N17_exit	New entry; post-update fetch fails (mock required)
PP10	N1->...->N11(F)->N12->N13(F)->N15->N16(F)->N18	New entry; checkout succeeds ($push path)

Edge	Definition	Notes
E1	N1 -> N2	
E2	N2 True -> N3	ValidationError raised
E3	N2 False -> N4	Validation passes
E4	N4 True -> N5	ObjectId parse failure
E5	N4 False -> N6	Valid ObjectId
E6	N6 True -> N7	Hardware document missing
E7	N6 False -> N8	Hardware found
E8	N8 True -> N9	Project document missing
E9	N8 False -> N10	Project found
E10	N10 True -> N10_exit	User not authorized
E11	N10 False -> N11	User authorized
E12	N11 True -> N11_exit	Insufficient availability
E13	N11 False -> N12	Sufficient availability
E14	N13 True -> N14	Existing entry – `$inc` path
E15	N13 False -> N15	New entry – `$push` path
E16	N14 -> N16	
E17	N15 -> N16	
E18	N16 True -> N17_exit	Post-update fetch failed (infeasible without mock)
E19	N16 False -> N18	Updated hardware fetched

Checkout/in Logic Coverage

- Predicate: c1 and c2
- c1: amount > 0
- c2: amount <= bound
- C1 major P=F is infeasible at guard nodes

Test	Major Clause	c1	c2	p	Status
T-CACC-CO-01	c2 major	T	T	T	Covered — checkout p=T
T-CACC-CO-02	c2 major	T	F	F	Covered — checkout p=F
T-CACC-CO-03	c1 major	F	–	F	INFEASIBLE at N11 — tested at N2 (Pydantic boundary)
T-CACC-CO-04	c2 boundary	T	T	T	Covered — boundary c2=(5<=5)=T
T-CACC-CI-01	c2 major	T	T	T	Covered — checkin p=T
T-CACC-CI-02	c2 major	T	F	F	Covered — checkin p=F
T-CACC-CI-03	c1 major	F	–	F	INFEASIBLE at N13 — tested at N2 (Pydantic boundary)

Input Domain Model

- Partition inputs.
  - Amount value
  - User assignment
  - Project existence

- Base choice coverage
- 8 test cases

Characteristic	Block	Label	Description
C1: `amount` value	b1	negative	amount < 0 — Pydantic rejects
C1: `amount` value	b2	zero	amount == 0 — Pydantic rejects
C1: `amount` value	b3 (base)	one	amount == 1 — minimum valid
C1: `amount` value	b4	exact capacity	amount == available — boundary valid
C1: `amount` value	b5	over capacity	amount > available — availability guard fires
C2: user assignment	b1 (base)	assigned	userId in assignedUsers
C2: user assignment	b2	not assigned	userId not in assignedUsers
C3: project existence	b1 (base)	valid	project found in DB
C3: project existence	b2	invalid	project not in DB

Checkout/in Results Table

Report File	Test Phase	Tests	Passed	Stmts	Miss	Branch	BrPart	Cover
`checkout-in-baseline-coverage.txt`	Baseline	8	8	153	74	44	10	**49%**
`checkout-in-cacc-coverage.txt`	CACC Logic	7	7	153	75	44	11	**48%**
`checkout-in-isp-coverage.txt`	ISP / BCC	8	8	153	76	44	10	**48%**
`checkout-in-structural-coverage.txt`	Structural (Graph)	12	12	153	72	44	4	**52%**

Phase	Tests	Cover	BrPart (partial branches)	Improvement
Baseline	8	49%	10	–
CACC Logic	7	48%	11	Exercises logic guard paths; slightly lower total due to smaller test count
ISP / BCC	8	48%	10	Exercises validation and availability partitions
Structural	12	**52%**	4	Highest coverage; partial branches reduced from 10→4 by exercising error paths

Takeaways (Conclusion)

- ISP was the highest-yield technique (7/7 defects)
- Structural/CACC contributed differently
- Mutation scores were numerically close but qualitatively different. Which mutants were killed matters
- Central argument: coverage percentages and mutation scores are useful indicators but not sufficient on their own; formal criteria make the reasoning behind gaps systematic and reproducible rather than accidental

.